

GlamLogistics — Boutique Delivery & Storefront Console App

Assignment 2 — Factory Method and Abstract Factory Course: ShP-2216 Software Design Patterns, Astana IT University

Project purpose

GlamLogistics is a console application for a fashion & beauty boutique. It plans the delivery of a boutique order by road (**Truck**) or by sea (**Ship**), and renders a matching storefront button and checkbox for either a **Windows** or a **macOS** admin panel.

- **Factory Method** creates the correct **Transport** (**Truck** / **Ship**) inside a **Logistics** creator hierarchy (**RoadLogistics** / **SeaLogistics**) and drives a shared **planDelivery(...)** workflow.
- **Abstract Factory** creates a matching pair of UI components (**Button** + **Checkbox**) for one platform family at a time (**WindowsFactory** / **MacOSFactory**), so the storefront never mixes a Windows button with a macOS checkbox.

Both patterns are wired together at startup in **Main** and used together by one **DeliveryApplication** client per run.

Package structure

```
src/main/java/com/glamlogistics/
├── transport/                Factory Method
│   ├── Transport.java       product interface
│   ├── Truck.java           concrete product (road)
│   ├── Ship.java            concrete product (sea)
│   ├── Logistics.java       abstract creator (factory method + shared workflow)
│   ├── RoadLogistics.java   concrete creator
│   └── SeaLogistics.java    concrete creator
├── ui/                       Abstract Factory
│   ├── Button.java          abstract product
│   ├── Checkbox.java        abstract product
│   ├── WindowsButton.java   concrete product (Windows family)
│   ├── WindowsCheckbox.java concrete product (Windows family)
│   ├── MacOSButton.java     concrete product (macOS family)
│   ├── MacOSCheckbox.java   concrete product (macOS family)
│   ├── GUIFactory.java      abstract factory
│   ├── WindowsFactory.java  concrete factory (Windows family)
│   └── MacOSFactory.java    concrete factory (macOS family)
└── app/
    ├── DeliveryApplication.java client: takes a GUIFactory + Logistics, uses both
    └── Main.java              startup: validates input, selects creator/factory
```

uml/ contains the two Graphviz source files and rendered PNG diagrams used in the report.

Prerequisites

- JDK 17
- Optional: Apache Maven 3.8+ (a plain `javac` / `java` build works too, see below)

Build and run

Option A — Maven

```
mvn clean compile
mvn exec:java -Dexec.mainClass="com.glamlogistics.app.Main" -Dexec.args="ROAD WINDOWS"
```

or build a runnable jar:

```
mvn clean package
java -jar target/glam-logistics.jar ROAD WINDOWS
```

Option B — plain `javac` / `java`

```
mkdir -p out
javac -d out $(find src -name "*.java")
java -cp out com.glamlogistics.app.Main ROAD WINDOWS
```

Supported input values

The application reads two independent values, first from command-line arguments (`arg[0]` = delivery mode, `arg[1]` = UI platform); any value missing or blank from the arguments is asked for interactively on the console instead:

Value	Supported inputs (case-insensitive)
Delivery mode	ROAD , SEA
UI platform	WINDOWS , MACOS

Any other value, or an empty/blank value at the console prompt, prints a clear validation message and stops the program cleanly (no crash, no silent default, no partial run).

Sample run

```
$ java -cp out com.glamlogistics.app.Main ROAD WINDOWS
Delivery mode: ROAD
UI platform: WINDOWS
Rendering Windows button: Confirm Order
Rendering Windows checkbox: Gift Wrap
Truck delivers satin evening dress collection to Almaty Flagship Boutique by road
```

Changing only the platform argument (**ROAD MACOS**) swaps the button/checkbox pair while still delivering by road; changing only the delivery argument (**SEA WINDOWS**) swaps the transport while keeping the same Windows storefront — the four combinations and the two invalid-input cases are recorded with full transcripts in the PDF report.

Author

- Full name: Serikbekkyzy Zhibek
- Group: SE-2523
- GitHub repository link: <https://github.com/z48284925-netizen/SDP-Assignment-2>